Course Introduction

이원형 교수
Dr. Lee, WonHyong
whlee@handong.edu

Course Management

- **Official Course Channel**
  - ✓ HDLMS → Announcements
- **KakaoTalk Team Chat**
  - ✓ https://invite.kakao.com/tc/ZUELq7dXj8
  - ✓ Note that it is an auxiliary communication channel.

ICE BREAKING

ICE BREAKING

Source: https://bbs.ruliweb.com/community/board/300143/read/69134758

Two Main Questions through This Semester

Why? **Can we do better?**

WHY

❖ **Why should I teach Algorithms Analysis to students?**

Why?

WHAT WE WILL LEARN

(OVERVIEW)

What We Will Learn

- **We want to solve a problem.**
  - ✓ Problem statement
  - ✓ Desired goal
- **We want to find a way (or ways) to solve the problem.**
  - ✓ Algorithm(s)
- **A way that is deterministic and efficient.**
  - ✓ Deterministic: generating the same output for the same input
  - ✓ Efficient: to solve problem within a feasible time bound with an input size
  - ✓ Algorithms Analysis
- **We do keep searching better ways.**
  - ✓ Design paradigms

❖ Key Question

- ✓ "Can we to better?"

❖ How to solve a problem

- ✓ Divide and conquer
  - Recurrence equation
    - » Substitution method
    - » Recursion-tree method
    - » Master method
- ✓ Dynamic programming
  - Memoization
    - » To avoid the repeated same calculation
    - » Bottom-up approach
  - Optimal substructure
  - Examples
    - » Parenthesization
    - » Longest common subsequence

√ Can we do better?

Growth of functions
$\Theta(\cdot)$, $O(\cdot)$ ⋯
to evaluate (time) complexity

❖ How to solve a problem

- ✓ Divide and conquer
- ✓ Dynamic programming
- ✓ Greedy Algorithms

√ Can we do better?

  - makes the choice that looks best at the moment
    - » Local optimum
  - for some (not all) optimization problems but in many cases
  - Examples
    - » Coin change
    - » Activity-selection problem
    - » Huffman codes
    - » Fractional Knapsack problem
      - But 0-1 Knapsack problem cannot be solved by greedy algorithm (even dynamic programming) → Branch and bound introduced

Growth of functions
$\Theta(\cdot)$, $O(\cdot)$ ⋯
to evaluate (time) complexity

❖ How to solve a problem

✓ Some problems can be described as **graphs**.

- Topological sort
- Finding connected component (undirected)
- Finding strongly connected component (directed)
- Critical path analysis
- Biconnected component (articulation point) problem
- Minimum spanning tree
  - » Kruskal's algorithm (greedy algorithm) $O(E \log V)$
  - » Prim's algorithm $O(E + V \log V)$
- Single-source shortest paths
  - » Bellman-Ford algorithm $O(VE)$
    - Negative edges allowed
  - » Applying topological sort (applying DFS) $\Theta(V + E)$
    - But constraints: directed acyclic graph
  - » Dijkstra's algorithm $O(E \log V)$ or $O(V \log V + E)$
    - But constraints: negative edges not allowed

Applying DFS
(cf. BFS)
$\Theta(V + E)$

√ Can we do better?

❖ How to solve a problem

✓ Some problems can be described as **graphs**.

- Single-source shortest paths
  - » Bellman-Ford algorithm $O(VE)$
    - Negative edges allowed
- All pair shortest paths
  - » Applying Bellman-Ford algorithm $|V|$ times $O(V^2E)$ or $O(V^4)$ (for a complex graph)
    - Negative edges allowed
  - » Floyd-Warshall Algorithm $\Theta(V^3)$
    - Applying dynamic programming!
    - Negative edges allowed

√ Can we do better?

❖ How to solve a problem

✓ We can also benefit from the **data structure**.
(case study: sorting algorithms)

- Insertion sort $O(n^2)$
  - » Sort in place
- Merge sort $\Theta(n \log n)$
  - » Doesn't sort in place
- Heap sort $O(n \log n)$
  - » Sort in place
  - » Heap property (data structure!)
- Quick sort $O(n \log n)$ (average case)
  - » Sort in place
  - » Not always better but faster in practice
- Counting sort $\Theta(n + k)$ or $\Theta(n)$ if $k = O(n)$
  - » But constraints: integer numbers in range 0 to k.
  - » It is a stable sort so that it can be used in Radix sort!
- Radix sort $\Theta(dn + dk)$ or $\Theta(n)$ if $k = O(n)$ and d is constant
  - » It can cover multiple digits. (but doesn't cover all real numbers)
- Bucket sort
  - » It covers real numbers in $[0,1]$ but assuming input is uniformly distributed then $\Theta(n)$.

√ Can we do better?

√ Can we do better?

√ Can we do better?

It is proved that
all of the comparison sorts
are $\Omega(n \log n)$.

√ BUT! can we do better?

√ Can we do better?

√ Can we do better?

❖ How to solve a problem

✓ We can also benefit from the **data structure**.
(case study: Finding order statistics and Dynamic set operations)

- Finding order statistics (finding the i-th smallest element of a set)
  - » Applying sorting algorithm and take i-th element $O(n \log n)$
  - » Using partition function from Quick sort $\Theta(n)$

√ Can we do better?

- Dynamic set operations
  - » Finding minimum, maximum, successor, predecessor
  - » Insertion, deletion

Use Max-heap (data structure)

√ Can we do better?

- Dynamic set operations
  - » Using Binary search tree
    - another data structure! more flexible, easy to search values
  - » But h could be n (worst case) or $\log n$ (best case)
    - We need balanced binary search tree!

√ Can we do better?

- Red-Black tree
  - » It keeps the balanced property (data structure!).
  - » Dynamic set operations can be done in $O(\lg n)$.

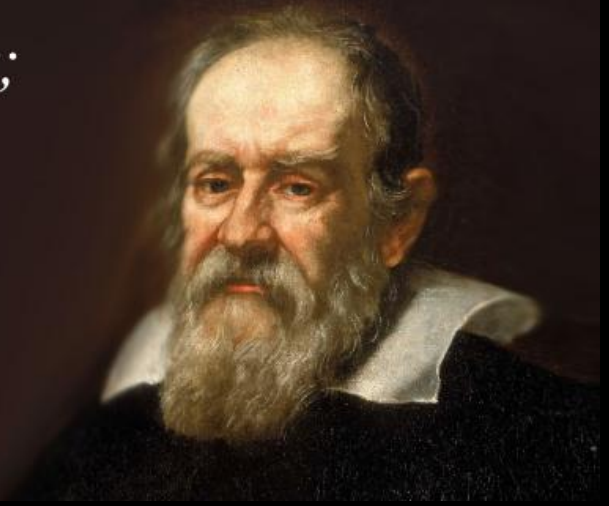
How to solve a problem

- ✓ We are still hungry…
- ✓ There are still bunch of unsolved problems.
  - If we find a deterministic algorithm that solves a problem in poly. time $O(n^k)$.
    - » We classify the problem as a $\mathbb{P}$ problem.
  - If we haven't found yet but there is a possibility to find a way to solve a problem,
    - » We classify the problem as a $\mathbb{NP}$ problem.
  - If it is proved that there is no way to solve a problem in polynomial time,
    - » The problem is no longer a $\mathbb{NP}$ problem.
  - Question
    - » "Is it worth to spend our time and resources to find ways to solve $\mathbb{NP}$ problems?"
    - » "What if it doesn't exist?"
  - Answer
    - » "We don't know yet." (This question itself is also an $\mathbb{NP}$ problem. LoL)
  - $\mathbb{NP}$-hard
    - » People found that there are some problems that every $\mathbb{NP}$ problems can be reducible to.
  - $\mathbb{NP}$-complete
    - » A problem that is both $\mathbb{NP}$-hard and $\mathbb{NP}$.
    - » So, if any $\mathbb{NP}$-complete is proved that it is a $\mathbb{P}$ problem, $\mathbb{NP}$ becomes $\mathbb{P}$ and it concludes that it is worth to spend our time and resources to find ways to solve $\mathbb{NP}$ problems.

- **How to solve a problem**
  - ✓ We are still hungry…
  - ✓ There are still bunch of unsolved problems.
  - ✓ How to solve $\mathbb{NP}$-hard problems that has exponential time complexity?
    - For sufficiently small input → applying exponential algorithm might be O.K.
    - Or Restrict input: isolate special cases that are solvable in polynomial time
    - Accept potentially incorrect solution (but solved in polynomial running time)
      - » Approximation algorithm
  - ✓ Algorithms for Quantum Computing

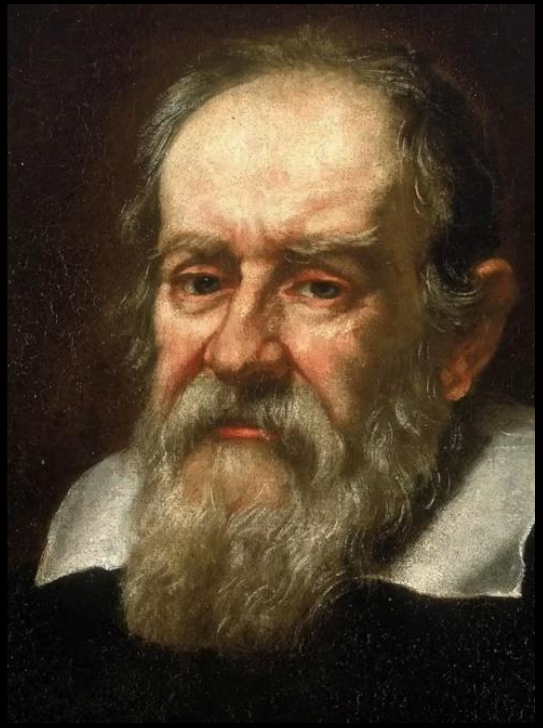
You cannot teach a man anything;
you can only help him find it
within himself.

Galileo Galilei



누구에게 무엇을 가르칠 수 없다
스스로 깨닫는 것을 도울 뿐이다

- 갈릴레오 갈릴레이 -



Course Philosophy

Our Grand Goals of This Course

❖ Course Philosophy

- ✓ Understanding algorithm principles over implementation speed
- ✓ Justification over output
- ✓ Responsibility over automation

- ✓ This course is not about how fast you can implement an algorithm.
- ✓ It is about whether you truly understand why an algorithm is needed, why it works, and when it should—or should not—be used.

- ✓ 이 수업은 코드를 빨리 짜는 능력을 평가하지 않을 예정이다.
- ✓ 왜 이 알고리즘이 필요한지, 왜 이것이 맞는지, 그리고 언제 쓰면 되는지 안 되는지를 이해하는 능력을 기르는 것에 목적이 있다.

How to Use AI

How to Use AI

- **AI Usage**
  - ✓ AI is NOT an answer generator.
  - ✓ But AI should be a thinking assistant.

I am responsible for my algorithm, not the AI.

Can we do better?

How to Use AI

❖ AI Usage Ground Rule

1. Use AI tools that allow conversation sharing
   - You may use ChatGPT, Gemini, or others, but your conversation history must be shareable via an online link.
2. MUST include a complete AI interaction history in your assignment submission.
   - Followings are required contents.
   - You may use Korean if you think it is more efficient, but the final answer must be written in English.
3. Explain Your Own Idea First to the AI
   - Clearly describe your approach and reasoning.
4. Get feedback from the AI and Verify AI's Response again
5. Request Alternative Algorithmic Ideas
   - Compare with your original idea.
6. Review the Implementation Process
   - Document modifications.
   - Explain why changes were necessary.
7. Study the AI-Generated Code
   - Understand structure and data choices
   - Get use to the coding convention.
8. Critically Analyze Strengths & Weaknesses

You remain responsible for the final answer.

Basic Course Information

COURSE MANAGEMENT

❖ Official Language

- ✓ English
  - Please use English in our class, your homework, exam, and all activities.
  - You will get NO POINTS in exam and homework if your final answers are not written in English

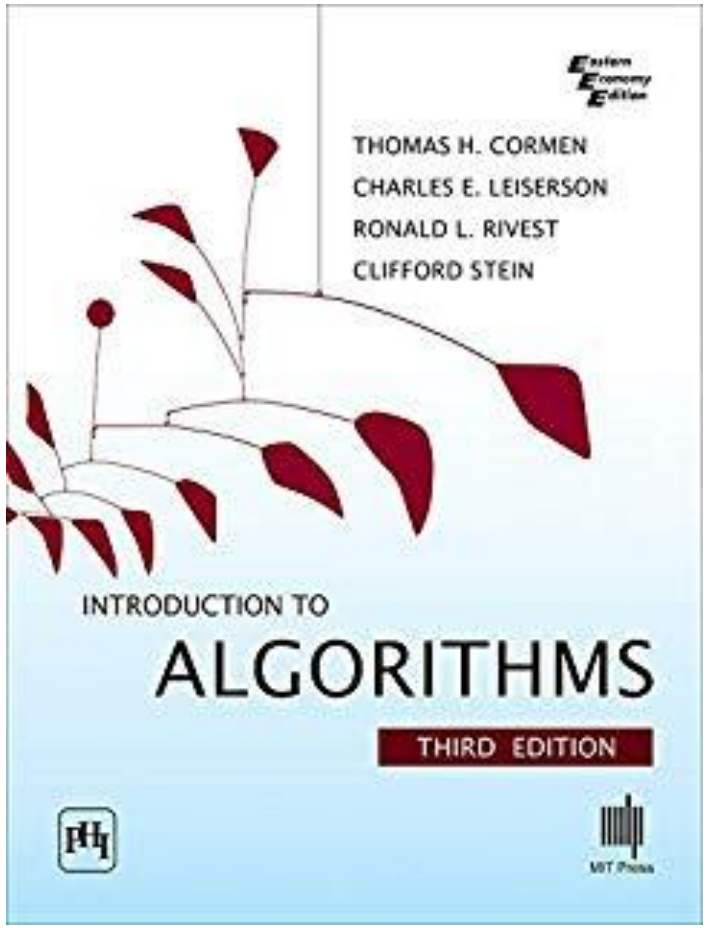
❖ Class Method

- ✓ On-site classes (default)
- ✓ Any alternative methods will be announced in advanced if necessary.

❖ The contents of the syllabus

- ✓ may be modified with or without notice according to the performance of the students, university events or other reasons.

- **Primary Textbook**
  - ✓ Introduction to Algorithms
    - Author: Thomas H. Cormen and Charles E. Leiserson
    - Publisher: The MIT press
    - Any Edition



TA

- **TA**
  - ✓ 조형민 CHO, HYEONGMIN
  - ✓ 22100713@handong.ac.kr

- **TA sessions would be provided upon requests.**

❖ HGU CSEE Standard & Honor Code Guideline

- ✓ 'F' grade will be given to students who attempt to cheat on exam/assignment/attendance.

- ✓ HGU Honor code and CSEE Standard apply.
  - https://lifove.github.io/files/HGU-CSEE-Standard_English_v0.3.pdf (English version)
  - https://lifove.github.io/files/HGU-CSEE-Standard_v0.3.pdf (Korean version)

- ✓ Scores can be corrected, but grades will never be negotiated.

❖ Grading Policies and Rates(%)

- ✓ Attendance
  - Randomly saying each student name at the beginning of each class
    Or confirming by assignment submission
  - **NOTE THAT** 'F' grade will be given to students who have missed equal to or greater than 25% (8 times) of the class or attempt to cheat their attendance.
  - If there is an unavoidable reason, the student is not considered absent only if the provided evidence shows that it complies with Article 34 of Academic Regulation.
    - » https://www.handong.edu/dcp/editor/files/rule_25.pdf
    - » http://rule.handong.edu/Gku.asp?Pun=3&Jang=3&Gku=1&Flag=SEARCH&Jo=34&Search_Word=#34 (The same but written in Korean)
- ✓ Midterm Exam 20%
- ✓ Final Exam 20%
- ✓ Quizzes 15%
- ✓ Assignments 35%
- ✓ Etc. 10%

Can we do better?

How to Evaluate

How to Evaluate

- **Base Letter Grade (A, B, C, etc.)**
  - ✓ Your total score is calculated from:
    - Midterm Exam 20%
    - Final Exam 20%
    - Quizzes 15%
    - Assignments 35%
    - Etc. 10% (This will be explained at the last page)
  - ✓ The base letter grade is determined through relative grading.

- **Plus(+) or Regular (A+ or A, B+ or B, etc.)**
  - ✓ After the base letter grade is assigned,
    the plus(+) or regular grade is determined based on your "Track Performance"

How to Evaluate

❖ Three Tracks

- ✓ Genius Track
  - You will solve more difficult problems,
  - participate actively in class discussions
- ✓ Transparency Track
  - If you use AI, you must document your thinking process with sharable links.
    If you do not use AI, you also must explain why.
    (Refer to the "How to Use AI" section)
  - Describe what you do not understand and what you are missing.
- ✓ Consistency Track
  - Submitting assignments on time.
  - Maintaining attendance.

❖ Track Selection

- ✓ You need to choose **two** out of the three tracks by 2:30pm on 3/13(Fri).
- ✓ You will have one chance to change your selection right after the midterm exam.

How to Evaluate

- **Three Tracks (Detailed criteria for receiving the "+" grade)**
  - ✓ Genius Track - Criteria
    - Solve bonus problems throughout the semester. For each assignment, submit a brief challenge reflection (3-5 lines), explaining why you chose to attempt the challenge (bonus problems) and what you learned even if you did not succeed.
      (You must solve at least 70% of the bonus problems and attempt at least 80% of them, including failures.)
    - Participate actively in class (asking questions, answering questions, or presenting solutions at the board) at least 12 times throughout the semester. (no more than two participations per class will be counted.)
  - ✓ Transparency Track - Criteria
    - For every assignment, for each question/problem you answer, you must clearly state whether AI was used and explain the reason for using (or not using) AI. If AI is used, include a link to the AI conversation history.
    - For every assignment, explicitly identify areas of uncertainty or parts you did not fully understand.
    - This must be done for at least 90% of your submitted answers throughout the semester.
  - ✓ Consistency Track - Criteria
    - Assignment scores do not influence this track evaluation.
    - Fewer than 5 late submissions. (A missing submission counts as two late submissions.)
    - Late submissions are accepted up to one week after the due date.
    - Blank submissions or submissions lacking substantial content will be treated as missing submissions.
    - Maintain attendance above 90%. (Two late arrivals equal one absence)
    - You can make up one late submission or one late arrival by submitting a weekly study report by the end of each week. (The report format is flexible, but supporting evidence of study activity is required. Late submission of weekly study reports will not be accepted.)

HOW TO EVALUATE

❖ Three Tracks ("+" 등급을 받기 위한 세부 기준)

✓ Genius Track - Criteria

- 학기 전체에 걸쳐 보너스 문제에 도전해야 한다. 그리고 각 과제마다 3–5줄 분량의 도전 기록을 제출해야 하며, 왜 해당 보너스 문제에 도전했는지, 그리고 무엇을 배웠는지를 성공하지 못했더라도 시도했다면 작성해야 한다. (보너스 문제의 최소 70% 이상을 해결해야 하고, 실패한 경우도 포함하여 80% 이상 시도해야 함.)
- 수업 시간에 적극적으로 참여해야 한다. (질문, 답변, 칠판 풀이 발표 등)
  학기 전체 동안 최소 12회 이상 참여해야 하며, 한 수업당 최대 2회까지만 인정한다.

✓ Transparency Track - Criteria

- 모든 과제에서, 각 문항(문제)별로 AI 사용 여부를 명확히 밝히고, 사용했거나 사용하지 않은 이유를 설명해야 한다. AI를 사용한 경우, 해당 AI 대화 기록 링크를 반드시 포함해야 한다.
- 모든 과제에서, 자신이 완전히 이해하지 못한 부분이나 불확실한 부분을 명시적으로 작성해야 한다.
- 위 사항은 학기 전체 제출 문항의 최소 90% 이상에서 충족되어야 한다.

✓ Consistency Track - Criteria

- 과제 점수는 이 트랙 평가에 영향을 미치지 않는다.
- 지각 제출이 5회 미만이어야 한다. (미제출은 지각 2회로 간주함.)
- 제출 마감일로부터 최대 1주일 후까지 제출이 가능하다.
- 내용이 없거나 부실한 제출물은 미제출물로 간주된다.
- 출석률이 90% 이상이어야 한다. (지각 2회는 결석 1회로 간주함.)
- 지각 제출 1회 또는 지각 출석 1회에 한하여, 주간 학습 보고서를 제출함으로써 만회할 수 있다. 보고서는 매주 해당 주 종료 전까지 제출해야 하며, 형식은 자유이나 학습 활동에 대한 증빙 자료를 포함해야 한다. 주간 학습 보고서의 지각 제출은 인정하지 않는다.

How to Evaluate

- **Track Selection Affects**
  - ✓ If you meet all the criteria of the two tracks you selected, you will receive a plus (+) grade.
  - ✓ If you meet the criteria of only one of the two tracks you selected, you will receive a regular grade.
  - ✓ If you meet none of the selected track criteria, you will receive a regular grade and no points for the "Etc. 10%" component. Otherwise, the full 10% will be given by default.

- **Track Selection (again)**
  - ✓ You need to choose **two** out of the three tracks by 4:00pm on 3/13(Fri).
  - ✓ You will have one chance to change your selection right after the midterm exam.

Choosing tracks is not about selecting an easier or harder path.
It is about choosing the habit you want to develop.

Q&A

Quiz 00

QUIZ 00

- **Find "Quiz 00" on HDLMS.**
  - ✓ Answer the question by 4:00pm on 3/13(Fri).
    - No worries. This quiz is simply for track selection. However, failure to complete it will result in significant penalties.

Thank you!